

MA3632 Lecture 11 – Pipelines and Reproducible Workflows

Throughout the first ten weeks of this module we have treated preprocessing and modelling as distinct activities carried out in sequence. In practice, however, this separation is dangerous. When a transformation is fitted on the full dataset before the train/test split, or when a transformation is fitted once on the full training dataset before cross-validation rather than being refitted within each training fold, the resulting error estimates can be optimistic. The solution is not greater care but better structure: a *pipeline* that enforces the correct order of operations automatically and treats the entire sequence – from raw features to predictions – as a single, reusable object.

This lecture formalises what a pipeline is, explains precisely how pipelines prevent preprocessing leakage, shows how heterogeneous feature types are handled via column transformers, and addresses the practical questions of hyperparameter search and persistence that arise when a pipeline is to be deployed.

1 The data leakage problem

Definition 1.1. *Data leakage* occurs when information from the validation or test portion of a dataset influences the fitting of a model or a preprocessing step, causing the reported generalisation error to be optimistic.

The most common form of leakage arises from fitting a transformation on the full dataset before splitting. Suppose one standardises each feature by subtracting its mean and dividing by its standard deviation. If the mean and standard deviation are computed on all n observations, the test set contributes to the statistics used to transform the training set. The model is then evaluated on data that have, in a small but real sense, already been seen.

A subtler version occurs during cross-validation. If standardisation is applied once to the full training set and then k -fold CV is used to estimate the error of a classifier, the validation fold within each split has been used to compute the scaler parameters. The fold is no longer truly held out.

Example 1.2. Consider a dataset of 1000 observations. A practitioner standardises the features using all 1000 samples, then performs 5-fold CV on a logistic regression. In each fold, 200 observations serve as the validation set. The mean and variance used for standardisation were computed in part from those 200 observations, so the validation fold is not independent of the transformation applied to the training fold. This can lead to an optimistically biased estimate of predictive performance, particularly in high-dimensional or small-sample settings.

The correct procedure is to fit any transformation *only on the training portion* of each split, and apply the fitted transformation to the validation portion without refitting. A pipeline enforces this automatically.

1.1 Leakage via feature selection

Scaling is one of the easiest forms of leakage to see, but feature selection can produce substantially more severe leakage. Such leakage occurs when features are *selected* using the full dataset before cross-validation begins. Suppose a dataset has $p \gg n$ features (as is common in genomics or text data), and a practitioner selects the k features most correlated with y using all n observations, then runs k -fold CV using only those k features.

Example 1.3. Take $n = 100$ observations and $p = 5000$ features, all pure noise independent of y . Selecting the 20 features most correlated with y on the full dataset, purely by chance, finds features that appear predictive. Cross-validating a classifier on those 20 features can report accuracy well above the true chance level of 50%, even though no feature carries any real signal. The leakage occurs because the validation fold’s labels were used, via the correlation ranking, to decide which features the model is even allowed to see.

This is structurally the same problem as scaler leakage: a step that should be refit on each training fold is instead fit once on the full dataset. The fix is identical in kind – place the selection step inside the pipeline (e.g. `SelectKBest` as a pipeline step) so that it is refit on the training fold within each CV split. Because feature selection can overturn the entire basis on which a model is judged, this form of leakage can produce a much larger optimism bias than scaler leakage, particularly when p is large relative to n .

Remark 1.4. This is not a hypothetical concern. Ambroise and McLachlan (2002) showed that several published studies using microarray gene-expression data had selected genes on the full dataset before cross-validating a classifier, producing reported error rates that were substantially optimistic; correcting the procedure to select genes within each fold raised the estimated error rates considerably.

1.2 Leakage from dependent observations

When observations are not independent – for example, multiple scans from the same patient, multiple sensor readings from the same device, or repeated purchases from the same customer – placing some of a group’s observations in the training fold and others in the validation fold leaks information through the shared identity, not through any preprocessing step.

Example 1.5. A medical imaging dataset contains five scans per patient, and a classifier is trained to detect a condition from a single scan. If patient scans are split randomly across training and validation folds rather than by patient, the model can partly memorise patient-specific anatomical characteristics from the training fold and exploit them when predicting on that same patient’s scans in the validation fold. The resulting CV accuracy substantially overstates how well the model would perform on scans from a genuinely new patient.

Remark 1.6. This form of leakage is *not* fixed by wrapping preprocessing in a pipeline, because it does not involve any transformer being fitted on the wrong data – it lies entirely in how the fold assignment is made. The fix is to change the cross-validation splitter itself, from a scheme that assigns observations to folds independently (e.g. `KFold`, `StratifiedKFold`) to one that keeps each group intact within a single fold, such as scikit-learn’s `GroupKFold`. Pipelines and correct splitters address two different failure modes, and both are necessary: a perfectly constructed pipeline evaluated with the wrong splitter still leaks.

2 Pipelines

Definition 2.1. A *pipeline* is an ordered sequence of transformers $T_1, T_2, \dots, T_{k-1}$ followed by a final estimator E . During fitting, each transformer T_i is fitted on the output of the preceding step and then transforms that output, in turn producing the input to T_{i+1} ; the final estimator E is fitted on the resulting representation after all $k - 1$ transformers have been applied. During prediction on new data, each already-fitted transformer applies its transformation without being refitted, and the final estimator predicts on the resulting representation.

The key property is that fitting and transforming are always coupled: within a single call to fit the pipeline, a transformer’s parameters are estimated only from the data supplied to that call. When the pipeline is used correctly – fitted fresh on each training fold, with prediction only ever applying already-fitted transformations to new data – this prevents leakage arising from a preprocessing step being fitted on validation data. It does not by itself prevent every form of leakage: as discussed in §1.2, leakage arising from how observations are assigned to folds in the first place is a separate failure mode that a pipeline alone cannot fix.

Remark 2.2. In scikit-learn, a pipeline is constructed as

```
from sklearn.pipeline import Pipeline
pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('classifier', LogisticRegression()),
])
```

Each step is a (name, object) pair. The names are arbitrary strings used to identify steps when setting hyperparameters. Calling `pipe.fit(X_train, y_train)` fits the scaler on `X_train`, transforms it, then fits the logistic regression on the transformed output. Calling `pipe.predict(X_test)` applies the *already fitted* scaler to `X_test` before passing it to the classifier.

Proposition 2.3. *Suppose a pipeline is used as the estimator passed to k -fold cross-validation, and no step in the pipeline accesses any data outside the current training fold (for example, no transformer is fitted before cross-validation begins on the full dataset). Then, within each fold, every data-dependent preprocessing parameter is fitted using only that fold’s training portion, and the validation observations for that fold do not influence any fitted parameter used to transform them.*

Sketch. In each fold, `cross_val_score` calls `pipe.fit` on the training fold and `pipe.score` on the validation fold. Because `fit` is called fresh on each training fold, every transformer is refitted using only the observations in that training fold, and `transform` on the validation fold reuses those already-fitted parameters without incorporating any information from the validation observations themselves. $\square$

3 Column transformers and heterogeneous features

Many real datasets contain a mixture of feature types: continuous measurements, ordinal categories, and nominal categories may require different preprocessing. A *column transformer* applies different transformation pipelines to different subsets of columns, then concatenates the results horizontally.

Definition 3.1. A *column transformer* is an object that partitions the feature matrix $X \in \mathbb{R}^{n \times p}$ into column subsets $X_{S_1}, X_{S_2}, \dots, X_{S_m}$ and applies transformer T_i to X_{S_i} . The outputs are concatenated column-wise to produce a transformed matrix of shape $n \times \sum_i q_i$, where q_i is the output dimension of T_i .

Example 3.2. A dataset contains three continuous features (age, income, years of experience) and two nominal categorical features (city, job title). A column transformer applies `StandardScaler` to the continuous columns and `OneHotEncoder` to the categorical columns, producing a single transformed matrix that is passed to the classifier. The entire object, including both transformers and the classifier, is wrapped in a pipeline so that cross-validation or a train/test split handles all steps correctly.

In scikit-learn, this pattern is expressed as follows.

Remark 3.3. `from sklearn.compose import ColumnTransformer`
`from sklearn.preprocessing import StandardScaler, OneHotEncoder`

```
preprocessor = ColumnTransformer([
    ('num', StandardScaler(),    ['age', 'income', 'experience']),
    ('cat', OneHotEncoder(),     ['city', 'job_title']),
])
```

```
pipe = Pipeline([
    ('prep',      preprocessor),
    ('classifier', RandomForestClassifier()),
])
```

Columns not listed in any transformer are dropped by default; the `remainder` parameter can be set to `'passthrough'` to retain them unchanged.

3.1 Imputation inside a pipeline

Missing value imputation is a transformation and must therefore sit inside the pipeline, not before it. Fitting a `SimpleImputer` on the full dataset before cross-validation leaks the column means (or medians) computed from the validation fold. The correct placement is as the first step of the numeric sub-pipeline within the column transformer.

Example 3.4. `from sklearn.impute import SimpleImputer`
`from sklearn.pipeline import Pipeline`
`from sklearn.compose import ColumnTransformer`

```
num_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler',  StandardScaler()),
])
```

```
preprocessor = ColumnTransformer([
    ('num', num_pipe,      numeric_cols),
    ('cat', OneHotEncoder(), categorical_cols),
])
```

The imputer is refitted on the training fold in each CV split, so the column medians are never contaminated by validation data.

4 Hyperparameter search over a pipeline

A pipeline's hyperparameters are named by combining the step name and the parameter name with a double underscore: `stepname__parametername`. This allows a grid or randomised search to tune preprocessing choices and model choices simultaneously.

Example 4.1. To search over the regularisation strength of a logistic regression and the imputation strategy simultaneously:

```
from sklearn.model_selection import GridSearchCV
```

```

param_grid = {
    'prep__num__imputer__strategy': ['mean', 'median'],
    'classifier__C':                  [0.01, 0.1, 1, 10],
}
search = GridSearchCV(pipe, param_grid, cv=5, scoring='accuracy')
search.fit(X_train, y_train)
print(search.best_params_)

```

Each combination of hyperparameters is evaluated by 5-fold CV, with all preprocessing steps refitted inside each fold.

Remark 4.2. Including preprocessing choices (imputation strategy, scaling method, encoding scheme) in the hyperparameter search is sometimes called *end-to-end tuning*. It correctly accounts for the interaction between preprocessing and model performance, and avoids the common mistake of selecting preprocessing steps based on intuition and then tuning only the model.

5 Pipeline persistence

Once a pipeline has been fitted, it encapsulates all the information needed to preprocess and predict on new data: the fitted scaler parameters, the encoder vocabulary, the model weights. This fitted object can be serialised to disk and reloaded later, making it the natural unit of deployment.

Definition 5.1. *Pipeline persistence* refers to saving a fitted pipeline object to disk so that it can be reloaded in a separate Python session and used for prediction without refitting.

A common approach for scikit-learn models uses the `joblib` library, which handles NumPy arrays efficiently.

Remark 5.2. `import joblib`

```

# After fitting:
joblib.dump(pipe, 'model_v1.joblib')

```

```

# In a new session (or deployment environment):
pipe_loaded = joblib.load('model_v1.joblib')
predictions = pipe_loaded.predict(X_new)

```

The loaded pipeline applies the *same* fitted transformations to `X_new` without refitting anything. This is the correct behaviour: a scaler fitted on 2024 training data should use the 2024 means and variances when scoring 2025 observations, not refit on the new data.

Remark 5.3. Serialised pipelines carry a version dependency: cross-version loading of a pipeline serialised under one scikit-learn version is not guaranteed to be supported reliably under a later version if the internal object structure has changed. It is good practice to record the library version alongside the saved file and to re-serialise when upgrading. The `sklearn` version can be recovered at serialisation time via `sklearn.__version__`.

6 Custom transformers

The scikit-learn pipeline interface is open: any compatible transformer implementing `fit` and `transform` can be inserted as an intermediate pipeline step. Classes inheriting from `TransformerMixin`

obtain a default `fit_transform` implementation. This allows domain-specific feature construction to sit inside the pipeline alongside standard preprocessing.

Definition 6.1. A *custom transformer* is a Python class that subclasses `BaseEstimator` and `TransformerMixin` from `sklearn.base`, implements `fit(self, X, y=None)` (which learns any necessary parameters from `X` and returns `self`), and implements `transform(self, X)` (which applies the learnt transformation and returns the result).

Example 6.2. A transformer that clips each feature to its interquartile range, with the clipping thresholds estimated from the training data:

```
from sklearn.base import BaseEstimator, TransformerMixin
import numpy as np
```

```
class IQRClipper(BaseEstimator, TransformerMixin):
    def fit(self, X, y=None):
        self.lower_ = np.percentile(X, 25, axis=0)
        self.upper_ = np.percentile(X, 75, axis=0)
        return self
    def transform(self, X):
        return np.clip(X, self.lower_, self.upper_)
```

This transformer can be inserted directly into a pipeline: its `lower_` and `upper_` attributes are fitted only on the training fold in each CV split, so extreme values in the validation fold do not influence the clipping thresholds.

7 Exercises

Exercise 7.1. A data scientist standardises all features on the full dataset, then performs 10-fold cross-validation to compare a logistic regression and a random forest. The logistic regression achieves a mean CV accuracy of 0.873 and the random forest achieves 0.901. She concludes that the random forest is superior and reports 0.901 as the expected test accuracy. Identify two methodological errors in this procedure and describe how each should be corrected.

Solution. The first error is fitting the scaler on the full dataset before splitting. The scaler computes column means and standard deviations from the full dataset, including observations that subsequently appear in each validation fold. The validation fold is therefore not independent of the preprocessing, and the CV accuracy is optimistic. A pipeline ensures the scaler is refitted on the training fold only in each CV split.

The second error is using the same CV evaluation to select between models and to estimate the error of the selected model. Choosing the model with the higher CV score and reporting that score as the expected test accuracy suffers from selection bias (see Lecture 10). The reported figure is optimistic because the winning score benefits from sampling luck. Nested CV, or a separate held-out test set evaluated only after selection, gives an honest estimate of the generalisation error of the selection procedure. $\square$

Exercise 7.2. Let $X \in \mathbb{R}^{n \times p}$ be a feature matrix with entries X_{ij} . A `StandardScaler` fitted on a training set X_{train} of size n_{train} computes, for each feature j ,

$$\hat{\mu}_j = \frac{1}{n_{\text{train}}} \sum_{i=1}^{n_{\text{train}}} X_{ij}, \quad \hat{\sigma}_j^2 = \frac{1}{n_{\text{train}}} \sum_{i=1}^{n_{\text{train}}} (X_{ij} - \hat{\mu}_j)^2,$$

and then transforms any matrix X' as $X'_{ij} \mapsto (X'_{ij} - \hat{\mu}_j)/\hat{\sigma}_j$.

Suppose instead that the scaler is fitted on the full dataset of size $n = n_{\text{train}} + n_{\text{test}}$, giving estimates $\tilde{\mu}_j$ and $\tilde{\sigma}_j$. Show that the difference $\tilde{\mu}_j - \hat{\mu}_j$ is proportional to the difference between the test-set mean and the training-set mean for feature j , and express the constant of proportionality in terms of n_{train} and n_{test} .

Solution. Write $\bar{X}_j^{\text{tr}}$ for the training mean and $\bar{X}_j^{\text{te}}$ for the test mean of feature j . The full-dataset mean is

$$\tilde{\mu}_j = \frac{n_{\text{train}} \bar{X}_j^{\text{tr}} + n_{\text{test}} \bar{X}_j^{\text{te}}}{n_{\text{train}} + n_{\text{test}}}.$$

Subtracting $\hat{\mu}_j = \bar{X}_j^{\text{tr}}$:

$$\tilde{\mu}_j - \hat{\mu}_j = \frac{n_{\text{test}}}{n_{\text{train}} + n_{\text{test}}} (\bar{X}_j^{\text{te}} - \bar{X}_j^{\text{tr}}).$$

The difference is therefore proportional to the difference between the test and training means, with constant of proportionality $n_{\text{test}}/(n_{\text{train}} + n_{\text{test}})$. When the test set is a small fraction of the data this difference is small, but it is not zero, and it is largest for features whose empirical means differ substantially between the training and test sets. $\square$

Exercise 7.3. A researcher has $n = 80$ observations and $p = 2000$ candidate features, all independent of y and of each other. She selects the 15 features with the highest absolute correlation with y , computed on all 80 observations, and then evaluates a logistic regression on those 15 features using 5-fold cross-validation.

Explain why the reported CV accuracy will be substantially above the true chance level of 50%, even though no feature carries any real signal. Describe precisely how the selection step should be modified so that the reported CV accuracy is an honest estimate, and state which scikit-learn construct achieves this automatically.

Solution. With $p = 2000$ independent noise features and only $n = 80$ observations, some features will show a strong sample correlation with y purely by chance, simply because 2000 independent draws give many opportunities for a large absolute correlation to occur even under the null. Selecting the top 15 by this criterion, using all 80 labels, means the selection step has already used the labels of every observation, including those that will later serve as the validation fold in each CV split. The subsequent cross-validation therefore evaluates a model on features that were chosen partly because they correlated with the very labels used to validate it, so the reported accuracy is optimistic – often well above 50% even though the true relationship is null.

The fix is to make feature selection part of the fitted procedure rather than a preprocessing step performed once: the selection criterion must be recomputed on the training fold only, inside each CV split, exactly as a scaler would be refit inside each fold. In scikit-learn this is achieved by inserting `SelectKBest` (or a similar selector) as a pipeline step before the classifier, so that `cross_val_score` or `GridSearchCV` refits the selector on each training fold automatically. $\square$

Exercise 7.4. A voice-based diagnostic classifier is trained on 40 short recordings from each of 30 patients (1200 recordings total), with the goal of detecting a speech disorder from a single recording of a previously unseen patient. A colleague uses ordinary 10-fold `StratifiedKFold` on the 1200 recordings and reports a cross-validated accuracy of 96%. Explain what is wrong with this evaluation, describe what scikit-learn construct would fix it, and state whether wrapping the existing preprocessing in a `Pipeline` object would be sufficient on its own to correct the problem.

Solution. With 40 recordings per patient, ordinary `StratifiedKFold` will typically place some recordings from every patient into both the training and the validation fold, since it assigns the 1200 recordings to folds independently of patient identity. A classifier can then partly learn patient-specific voice characteristics (pitch, accent, recording conditions) from the training recordings and use them to correctly classify the same patient’s held-out recordings, without having learned anything that generalises to a new patient. The reported 96% therefore substantially overstates performance on a genuinely unseen patient, whose voice characteristics the model has never encountered in any form.

The correct construct is `GroupKFold` (or `StratifiedGroupKFold`, if class balance across folds is also required), passing the patient identifier as the `groups` argument, so that all 40 recordings from a given patient are placed entirely within one fold.

Wrapping the preprocessing in a `Pipeline` is not sufficient on its own: the leakage here does not arise from a transformer being fitted on validation data, but from the choice of which observations are assigned to which fold. A perfectly constructed pipeline evaluated with `StratifiedKFold` would still leak in exactly this way; the splitter itself must be changed to `GroupKFold`. $\square$

Further Reading

Kaufman, Rosset, Perlich, and Stitelman (2012) give a systematic treatment of leakage in data mining, covering both the scaler-style leakage and the feature-selection leakage discussed in this lecture, with real case studies from industry. Ambroise and McLachlan (2002) is the classic worked example of feature-selection leakage, showing the severe selection bias that can arise when feature selection is performed before cross-validation and how feature selection should instead be incorporated into the validation procedure. Roberts et al. (2017) give a systematic treatment of the group-leakage problem discussed in §1.2, covering temporal, spatial, hierarchical, and grouped dependence structures and the corresponding block-style cross-validation schemes needed to avoid overstating generalisation performance. The scikit-learn user guide sections on `Pipeline`, `ColumnTransformer`, and custom transformers give the authoritative reference for the API used throughout this lecture. Lindholm et al. (*SML*, smlbook.org) and James et al. (*ISLP*, statlearning.com) both discuss the train/validation/test protocol that pipelines are designed to enforce automatically.

References

- Ambroise, C. and McLachlan, G.J., 2002. Selection bias in gene extraction on the basis of microarray gene-expression data. *Proceedings of the National Academy of Sciences*, 99(10), pp.6562–6566.
- James, G., Witten, D., Hastie, T., Tibshirani, R. and Taylor, J., 2023. *An Introduction to Statistical Learning with Applications in Python*. Springer.
- Kaufman, S., Rosset, S., Perlich, C. and Stitelman, O., 2012. Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data (TKDD)*, 6(4), pp.1–21.
- Lindholm, A., Wahlström, N., Lindsten, F. and Schön, T.B., 2022. *Machine Learning: A First Course for Engineers and Scientists*. Cambridge University Press.
- Roberts, D.R., Bahn, V., Ciuti, S., Boyce, M.S., Elith, J., Guillera-Arroita, G., Hauenstein, S., Lahoz-Monfort, J.J., Schröder, B., Thuiller, W. and Warton, D.I., 2017. Cross-validation

strategies for data with temporal, spatial, hierarchical, or phylogenetic structure. *Ecography*, 40(8), pp.913–929.

scikit-learn developers. *Pipelines and Composite Estimators*. scikit-learn user guide, scikit-learn.org.